Universidad de San Carlos de Guatemala
Centro Universitario de Occidente División de Ciencias de la Ingeniería
Laboratorio de Base de Datos 2
Aux. Emilio Maldonado
Sección: A



Manual Tecnico

Carlos Raúl Alberto López Peláez 202031871

Quetzaltenango, 30 de Octubre del 2026

1. Arquitectura General del Sistema

El sistema implementa una arquitectura cliente-servidor. El cliente web se comunica mediante peticiones HTTP/JSON hacia el prefijo /api/* del servidor de aplicación , con sesiones y pools de conexión de MongoDB y Neo4j dentro de la capa de servidor.

En la siguiente tabla se resumen la arquitectura del sistema, sus tecnologías principales, puertos de red y responsabilidades técnicas:

Capa / Componente	**Tecnología y Versión**	**Puerto**	**Responsabilidad Técnica Principal**
Frontend SPA	Vue 3.5 + TypeScript 5.9 + Vite 8 + Chart.js 4.6	5173	Renderizado reactivo del Dashboard, gráficas multivista para P1–P20, lienzo interactivo de grafos , carga CSV y benchmark.
Backend API REST	NestJS 10.4 + Express + TypeScript 5.7	3000	Pipeline ETL, validación de DTOs, ejecución de pipelines de agregación MongoDB y consultas Cypher, manejo global de errores.
Motor Documental	MongoDB 7.0	27017	Almacenamiento primario de evaluaciones, bitácoras de importación, auditoría de incidencias CSV y agregaciones analíticas P1–P20.
Motor de Grafos	Neo4j 5.24 Community	7474 / 7687	Modelado topológico con 8 etiquetas de nodos y 10 relaciones dirigidas para consultas de recorrido multi-salto G1–G5 y ego-grafos.

2. Diseño de las Bases de Datos

2.1 Base de Datos Documental: MongoDB 7.0

En MongoDB se utiliza la base de datos usac_pruebas_especificas, estructurada en cuatro colecciones especializadas que separan los hechos académicos validados, la trazabilidad de cargas, los registros con anomalías y las vistas materializadas:

Colección	**Rol en la Arquitectura**	**Volumen / Naturaleza de Datos**
evaluaciones	Colección principal de hechos. Cada documento representa el intento de evaluación de un aspirante en una materia y fecha específica.	208,453 documentos válidos . Escritura por lotes.
importaciones	Bitácora transaccional de ejecuciones del pipeline ETL. Registra tiempos, estado de sincronización con Neo4j y contadores detallados.	1 documento por cada archivo CSV procesado.
errores_importacion	Almacena cada fila rechazada o marcada con advertencia junto con su contenido original.	1 documento por incidencia detectada.
resumen_carrera	Colección auxiliar materializada de la etapa $merge de la consulta P6.	1 documento consolidado por cada carrera objetivo de la Facultad de Ingeniería.

2.1.1 Esquema del Documento en la Colección evaluaciones

Cada documento almacenado en la colección evaluaciones obedece la interfaz EvaluacionDocument. Para garantizar la idempotencia ante reimportaciones del mismo archivo CSV, el sistema calcula una huella determinista (evaluacion_uid) combinando los atributos clave del evento:

Campo	**Tipo de Dato**	**Descripción Técnica y Regla de Dominio**
evaluacion_uid	string (UNIQUE)	Identificador determinista único del evento de evaluación para evitar duplicados en reimportaciones.
correlativo	string	Identificador del aspirante durante el proceso de admisión (ej. correlativo de asignación).
carnet	string | null	Número de carnet estudiantil cuando el registro cuenta con inscripción formal; null si no aplica.
fecha_asignacion	string (YYYY-MM-DD)	Fecha normalizada en formato ISO corto en que se asignó la prueba específica.
anio_asignacion / mes_asignacion	number	Año y mes numérico extraídos de fecha_asignacion
fecha_evaluacion	string | null	Fecha de aplicación del examen cuando está disponible en el archivo de origen.
sexo	'F' | 'M' | 'NO_ESPECIFICADO'	Sexo normalizado del aspirante tras homologación léxica.
edad / anio_nacimiento	number | null	Edad calculada al momento de la prueba
grado_academico	string	Título o grado de educación media reportado por el aspirante.
carrera_objetivo	string	Nombre homologado en mayúsculas de la carrera de Ingeniería a la que aspira.
establecimiento	string	Nombre normalizado del colegio o instituto de educación media de procedencia.
departamento_establecimiento	string	Departamento geográfico donde se ubica el establecimiento educativo.
municipio_establecimiento	string	Municipio geográfico del establecimiento educativo.
municipio_uid	string	Clave compuesta DEPARTAMENTO::MUNICIPIO.
tipo_establecimiento	string (Enum)	Clasificación institucional: PUBLICO,

		PRIVADO, POR COOPERATIVA, MUNICIPAL o SIN_TIPO_ESTABLECIMIENTO.
materia_evaluada	string	Asignatura evaluada en la prueba específica
numero_evaluacion	number	Número de oportunidad o intento del examen específico en el año
anio_evaluacion	number	Año lectivo al que corresponde la evaluación específica.
resultado_evaluacion / aprobado	string / boolean	Texto oficial ('APROBADO' | 'NO APROBADO') y bandera booleana (true | false) para agregaciones rápidas.
advertencias	string[]	Arreglo de códigos de advertencia no fatales asociados al documento
importacion_id / fila_origen	string / number	UUID de la corrida de importación y número de línea en el archivo CSV original para trazabilidad.

2.1.2 Indexación en MongoDB

Al arrancar el servidor backend, el servicio MongoSchemaService ejecuta el método indexandoMongo(), creando 11 índices distribuidos en las 4 colecciones para optimizar las consultas P1–P20, la deduplicación por lotes y la búsqueda de aspirantes:

Colección	**Nombre del Índice**	**Definición de Claves**	**Propósito y Consultas Optimizadas**
evaluaciones	idx_evaluacion_uid_unique	{ evaluacion_uid: 1 } (UNIQUE)	Garantiza e impide duplicar evaluaciones.
evaluaciones	idx_correlativo_historial	{ correlativo: 1, fecha_asignacion: 1, numero_evaluacion: 1 }	Acelera la reconstrucción del historial cronológico de un aspirante y conteo de intentos.
evaluaciones	idx_carnet	{ carnet: 1 }	Optimiza la búsqueda puntual y filtrado de aspirantes por número de carnet.
evaluaciones	idx_anio_materia_aprobado	{ anio_evaluacion: 1, materia_evaluada: 1, aprobado: 1 }	Cubre reprobación y aprobación por materia y año
evaluaciones	idx_carrera_anio_aprobado	{ carrera_objetivo: 1, anio_evaluacion: 1, aprobado: 1 }	Acelera indicadores de rendimiento por carrera objetivo

evaluaciones	idx_tipo_mes_materia	{ tipo_establecimiento: 1, anio_asignacion: 1, mes_asignacion: 1, materia_evaluada: 1 }	Soporta series temporales mensuales y comparativas por sector educativo.
evaluaciones	idx_ubicacion_carrera	{ departamento_establecimiento: 1, municipio_establecimiento: 1, carrera_objetivo: 1 }	Optimiza análisis geográficos por departamento, municipio y establecimiento.
importaciones	idx_importacion_id_unique	{ id: 1 } (UNIQUE)	Búsqueda directa de po UUID.
importaciones	idx_importacion_created_at	{ created_at: -1 }	Ordenamiento descendente del historial de cargas CSV.
errores_importacion	idx_importacion_tipo_fila	{ importacion_id: 1, tipo: 1, fila: 1 }	Paginación filtrada de errores y advertencias por corrida de importación.
resumen_carrera	idx_resumen_carrera_unique	{ carrera_objetivo: 1 } (UNIQUE)	Requerido por $merge de P6 para actualizar atómicamente el resumen por carrera.

2.2 Base de Datos Orientada a Grafos: Neo4j 5.24

El servicio Neo4jSchemaService asegura al inicio del sistema la creación de 8 restricciones de unicidad, las cuales crean automáticamente índices de búsqueda que garantizan que las operaciones MERGE en lotes sean rápidas.

2.2.1 Catálogo de Nodos y Restricciones

Etiqueta de Nodo (Label)	**Propiedad Clave (UNIQUE)**	**Restricción en Neo4j**	**Propiedades Almacenadas en el Nodo**
:Aspirante	correlativo	aspirante_correlativo_unique	correlativo, carnet, sexo, edad, grado_academico
:Evaluacion	evaluacion_uid	evaluacion_uid_unique	evaluacion_uid, correlativo, fecha_asignacion, anio_asignacion, mes_asignacion, fecha_evaluacion, numero_evaluacion, anio_evaluacion, resultado_evaluacion, aprobado, edad_evaluacion
:Materia	nombre	materia_nombre_unique	nombre
:Carrera	nombre	carrera_nombre_unique	nombre
:Establecimiento	nombre	establecimiento_nombre_unique	nombre, tipo, departamento, municipio
:TipoInstitucion	nombre	tipo_institucion_nombre_unique	nombre
:Municipio	municipio_uid	municipio_uid_unique	municipio_uid , nombre, departamento
:Departamento	nombre	departamento_nombre_unique	nombre

2.2.2 Catálogo de Relaciones Dirigidas del Grafo

La proyección ejecutada por proyectarEvaluacionesNeo4j conecta los 8 tipos de nodos mediante 10 relaciones dirigidas que habilitan tanto recorridos centrados en la evaluación como recorridos institucionales y geográficos:

Nodo Origen	**Relación Dirigida**	**Nodo Destino**	**Semántica en el Dominio y Uso en Consultas G1–G5**
(:Aspirante)	[:REALIZO]	(:Evaluacion)	Vincula al aspirante con cada evento de evaluación sustentado
(:Aspirante)	[:ESTUDIO_EN]	(:Establecimiento)	Conecta directamente al aspirante con su colegio o instituto de egreso
(:Evaluacion)	[:EVALUA_MATERIA]	(:Materia)	Indica qué asignatura específica fue evaluada en dicho intento
(:Evaluacion)	[:ASPIRA_A]	(:Carrera)	Asocia la evaluación con la carrera objetivo solicitada por el aspirante
(:Evaluacion)	[:PROCEDE_DE_ESTABLECIMIENTO]	(:Establecimiento)	Trazabilidad directa del establecimiento reportado en el evento de evaluación.
(:Evaluacion)	[:PROCEDE_DE_TIPO]	(:TipoInstitucion)	Vincula la evaluación con el sector educativo del establecimiento.
(:Evaluacion)	[:PROCEDE_DE_MUNICIPIO]	(:Municipio)	Asocia la evaluación con el municipio de procedencia .
(:Establecimiento)	[:DE_TIPO]	(:TipoInstitucion)	Clasifica al establecimiento dentro de su sector institucional.
(:Establecimiento)	[:UBICADO_EN]	(:Municipio)	Ubica geográficamente al establecimiento dentro de su municipio .
(:Municipio)	[:PERTENECE_A]	(:Departamento)	Jerarquía territorial entre municipio y departamento de Guatemala .

3.Backend (NestJS)

El servidor Backend está construido sobre NestJS 10 bajo el paradigma de inyección de dependencias, modularidad estricta y tipado estático con TypeScript 5.7. A continuación se el flujo de inicialización, arquitectura interna y diseño por módulos.

3.1 Inicialización, Seguridad y Manejo Global de Errores

- **Prefijo Global y CORS:** Todas las rutas se exponen bajo el prefijo /api. El middleware CORS valida los orígenes permitidos según la variable de entorno CORS_ORIGIN y habilita los métodos GET, POST, PUT, PATCH, DELETE y OPTIONS.
- **Validación de Variables de Entorno:** Al iniciar AppModule, ConfigModule valida la presencia y formato de PORT, CORS_ORIGIN, MONGODB_URI, MONGODB_DB_NAME, NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD y MAX_CSV_FILE_SIZE_MB antes de levantar los adaptadores de red.

- **Filtro Global de Excepciones :** Intercepta cualquier excepción HTTP o error no controlado y estandariza la respuesta JSON con los campos statusCode, timestamp, path y error (incluyendo message y details), evitando filtrar trazas internas del servidor al cliente.

3.2 Arquitectura Funcional de los Módulos del Backend

Módulo NestJS	**Controladores y Servicios Clave**	**Responsabilidad Técnica Detallada**
DatabaseModule (@Global)	DatabaseService, MongoSchemaService, Neo4jSchemaService	Provee globalmente los tokens de inyección MONGODB_CLIENT, MONGODB_DB y NEO4J_DRIVER.
HealthModule	HealthController, HealthService	Ejecuta pruebas activas de conectividad midiendo la latencia en milisegundos de cada motor.
ImportsModule	ImportsController, ImportsService, procesarCsv, proyectarEvaluaciones Neo4j	Gestiona la recepción de archivos CSV , autodetección de codificación UTF-8/Windows-1252, normalización léxica, escritura por lotes en MongoDB y sincronización por lotes en Neo4j). Incluye endpoints para auditar errores y vaciar o resincronizar las bases.
AnalyticsModule	AnalyticsController, AnalyticsService, paginarConsulta	Implementa los resúmenes del Dashboard y las 20 consultas . La utilidad paginarConsulta inyecta dinámicamente etapas de búsqueda ($match), ordenamiento ($sort), conteo y paginación ($skip, $limit), y serializa el pipeline exacto en la propiedad

		mongoQuery.
ApplicantsModule	ApplicantsController, ApplicantsService	Provee los catálogos de filtrado, la búsqueda paginadas de aspirantes agrupados por correlativo y la vista de expediente individual con métricas de aprobación e historial cronológico de evaluaciones.
GraphsModule	GraphsController, GraphsService, BenchmarkService	Ejecuta las 5 consultas topológicas en Cypher transformando los resultados en estructuras de visualización y ejecuta el benchmark comparativo midiendo tiempos de respuesta entre MongoDB y Neo4j.

4. Librerías y Dependencias Utilizadas

A continuación se documentan todas las librerías de producción y desarrollo utilizadas tanto en el Backend como en el Frontend, especificando su versión en package.json y su justificación técnica dentro del proyecto.

4.1 Librerías del Backend (backend/package.json)

Librería / Paquete	**Versión**	**Tipo**	**Uso Técnico y Justificación en el Backend**
@nestjs/core / @nestjs/common	^10.4.15	Producción	Núcleo del framework NestJS: contenedor de inversión de control (IoC), inyección de dependencias, decoradores de controladores, pipes, filtros y ciclo de vida.
@nestjs/platform-express	^10.4.15	Producción	Adaptador HTTP basado en Express y middleware Multer (FileInterceptor) para procesar cargas de archivos CSV en formato multipart/form-data.
@nestjs/config	^3.3.0	Producción	Carga y validación tipada de variables de entorno (.env) mediante el servicio inyectable ConfigService.
mongodb	^6.12.0	Producción	Driver nativo oficial de MongoDB para Node.js. Se eligió sobre Mongoose para ejecutar pipelines de agregación complejos (P1–P20) y operaciones bulkWrite sin sobrecarga de hidratación de objetos.
neo4j-driver	^5.27.0	Producción	Driver oficial de Neo4j sobre el protocolo binario Bolt. Permite ejecutar sesiones transaccionales y consultas Cypher parametrizadas (G1–G5 y proyecciones UNWIND).
csv-parse	^5.6.0	Producción	Parser CSV de alto rendimiento conforme al estándar RFC 4180, utilizado en el ETL para tokenizar filas con soporte de comillas escapadas y delimitadores variables.
class-validator	^0.14.1	Producción	Validación declarativa basada en decoradores (@IsOptional, @IsInt, @Min, @Max, @IsString) aplicada sobre los DTOs de entrada de la API.
class-transformer	^0.5.1	Producción	Transformación e instanciación automática de objetos planos HTTP a

			instancias de clases DTO tipadas.
rxjs / reflect-metadata	^7.8.1 / ^0.2.2	Producción	Programación reactiva para interceptores de NestJS y soporte de reflexión de metadatos de decoradores en TypeScript.
typescript	^5.7.2	Desarrollo	Compilador de TypeScript con verificación estricta de tipos para todo el código del servidor.
jest / ts-jest / supertest	^29.7.0 / ^7.0.0	Desarrollo	Ejecución de pruebas unitarias y de integración HTTP sobre los controladores, servicios y utilidades de validación CSV.

4.2 Librerías del Frontend

Librería / Paquete	**Versión**	**Uso Técnico y Justificación en el Frontend**
vue	^3.5.44	Framework progresivo para la interfaz de usuario utilizando Composition API (<script setup lang='ts'>) y reactividad basada en proxies.
vue-router	^4.6.4	Enrutador oficial de la SPA para gestionar la navegación sin recarga entre las 5 vistas principales (/, /carga, /aspirantes, /grafos y /comparacion).
chart.js / vue-chartjs	^4.6.1 / ^5.3.4	Motor de renderizado en HTML5 Canvas para las gráficas del Dashboard (SampleChart.vue) y las visualizaciones dinámicas de las consultas P1–P20 (QueryResultChart.vue: barras, líneas, área y dona).
chartjs-plugin-zoom / hammerjs	^2.2.0 / ^2.0.8	Soporte de gestos táctiles, zoom interactivo y desplazamiento (panning) sobre los gráficos estadísticos de series extensas.
vis-network	^10.1.3	Soporte de estructuras y layouts de redes complementario al lienzo interactivo SVG de grafos (GraphCanvas.vue).
reka-ui / @vueuse/core	^2.11.0 / ^15.0.0	Primitivas de interfaz accesibles y utilidades reactivas de navegador para manejo de estados, eventos y dimensiones de ventana.
vite / @vitejs/plugin-vue	^8.3.4 / ^6.0.10	Empaquetador web de última generación con Hot Module Replacement (HMR) instantáneo y compilación optimizada de componentes .vue.
typescript / vue-tsc	^5.9.3 / ^3.3.13	Verificación estática de tipos TypeScript tanto en

		módulos de servicio como dentro de las plantillas de los componentes Vue.

5. Pipeline ETL, Limpieza y Sincronización

El flujo ETL implementado en ImportsService procesa archivos CSV de hasta 50 MB aplicando decodificación automática UTF-8 / Windows-1252, limpieza de comillas y espacios redundantes, homologación de textos a MAYÚSCULAS sin tildes y clasificación de cada fila antes de persistirla:

Regla de Dominio (Centinela EDAD = 92): En el dataset de 2022 existen 1,704 registros con EDAD = 92 utilizados como valor centinela ante ausencia de fecha de nacimiento. El ETL preserva la evaluación como válida, registra una ADVERTENCIA (EDAD_ANOMALA_NO_PROMEDIABLE) en la colección errores_importacion y asigna edad = null en el documento de evaluaciones, evitando distorsionar los promedios de edad en P4 y P11.

Para maximizar el rendimiento durante cargas masivas, MongoDB ejecuta operaciones bulkWrite con updateOne ({ evaluacion_uid }, { $setOnInsert: doc }, { upsert: true }) en bloques de 500 documentos, mientras que Neo4j sincroniza el grafo mediante sentencias Cypher parametrizadas con UNWIND $rows en lotes de 500 registros usando cláusulas MERGE idempotentes.

6. Requisitos, Configuración y Despliegue Paso a Paso

6.1 Prerrequisitos

- **Node.js** >= 20.x y npm >= 10.x
- **Docker** y Docker Compose v2

6.2 Variables de Entorno (backend/.env)

Copie el archivo .env.example en .env y backend/.env con la siguiente configuración:

```
PORT=3000
CORS_ORIGIN=http://localhost:5173
MONGODB_URI=mongodb://[::1]:27017
MONGODB_DB_NAME=usac_pruebas_especificas
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=pfb2_neo4j_password
MAX_CSV_FILE_SIZE_MB=50
```

6.3 Ejecución de Contenedores, Backend y Frontend

```
# 1. Levantar contenedores de MongoDB y Neo4j
docker compose --env-file backend/.env up -d

# 2. Instalar dependencias e iniciar el Backend (puerto 3000)
cd backend
npm install
npm run start:dev

# 3. Instalar dependencias e iniciar el Frontend (puerto 5173)
cd frontend
npm install
npm run dev
```

Nota sobre Windows y el puerto 27017: Si el sistema operativo anfitrión tiene activo el servicio nativo de Windows MongoDB sobre 127.0.0.1:27017, deténgalo con Stop-Service -Name MongoDB o utilice la URI mongodb://[::1]:27017/usac_pruebas_especificas para conectarse directamente al contenedor Docker pfb2-mongodb.

7. Catálogo de Endpoints de la API REST (/api)

Todos los endpoints de la API devuelven estructuras JSON tipadas. En las 20 consultas analíticas de MongoDB se retorna el contrato PaginaConsulta<T> con items, total, page, limit, totalPages y mongoQuery (la sentencia db.<coleccion>.aggregate([...]) ejecutada):

Módulo	**Método y Ruta**	**Descripción Técnica del Endpoint**
Health	GET /api/health	Diagnóstico en vivo de disponibilidad y latencia (ms) de API, MongoDB y Neo4j.
Imports	POST /api/imports/csv	Ingesta multipart/form-data de CSV, limpieza ETL, carga Mongo y sync Neo4j.
Imports	GET /api/imports	Historial paginado de importaciones ejecutadas con contadores de auditoría.
Imports	GET /api/imports/:id/errors	Detalle paginado de filas rechazadas o con advertencia para una importación.
Imports	POST /api/imports/resync-neo4j	Reconstrucción total o por importacionId del grafo en Neo4j desde MongoDB.
Imports	DELETE /api/imports/clear	Vaciado completo de las colecciones en MongoDB y del grafo en Neo4j.
Analytics	GET /api/analytics/summary	KPIs anuales y serie mensual de evaluaciones totales vs aprobadas.
Analytics	GET /api/analytics/period	KPIs filtrados dinámicamente por granularidad mensual, semanal o diaria.
Analytics	GET /api/analytics/queries/p1..p20	Ejecución paginada, buscable y ordenable de las 20 agregaciones MongoDB + mongoQuery.
Analytics	POST /api/analytics/resumen-carrera	Materializa mediante $merge la colección auxiliar resumen_carrera (P6).
Applicants	GET /api/applicants/filters	Catálogos distintos (años, carreras, materias, tipos, departamentos) para filtros.
Applicants	GET /api/applicants	Búsqueda multicriterio paginada de aspirantes en MongoDB.
Applicants	GET /api/applicants/:correlativo	Perfil consolidado e historial cronológico de evaluaciones de un aspirante.
Graphs	GET /api/graphs/filters	Catálogos disponibles en Neo4j para parametrizar subgrafos.
Graphs	GET /api/graphs/queries/g1..g5	Ejecuta las consultas Cypher G1–G5 devolviendo nodos, aristas, tabla y cypher.

Graphs	GET /api/graphs/benchmark	Ejecuta el benchmark comparativo MongoDB vs Neo4j para los escenarios BM-01 a BM-04.